$ ./jvm run X.class

Projeto lógico da JVM

Software Básico, UnB

C99 · GCC

Antonio Coelho · Daniel Morais · Gabriel Pinto · Leonardo Mileo · Lucas Mendes

jvm — laço principal

```
while (vm->frame) {
    Frame *f = vm->frame;
    u1 op = f->method->code[f->pc];
    Handler h = OPCODES[op].handler;
    if (!h) erro_opcode_invalido(op);
    h(vm, f, op);
}
```

JÁ IMPLEMENTADO

Leitor-exibidor de .class

- Parser completo do .class, com leitura big-endian e checagem de limites
- Pool de constantes com índices e valores resolvidos
- Tabela dos 202 opcodes, com mnemônico e formato dos operandos
- Code, SourceFile, ConstantValue, Signature, Exceptions, LineNumberTable e LocalVariableTable
- cppcheck, ASan/UBSan e libFuzzer

dois modos, o mesmo parser

```
$ ./jvm view X.class
exibidor, já implementado

$ ./jvm run X.class
execução, a implementar
```

VISÃO GERAL

O slot é a unidade básica

Cada posição das variáveis locais e da pilha de operandos é um Slot de 8 bytes.

- A instrução define o campo: iadd lê .i, dadd lê .d
- A referência é um ponteiro C direto no slot
- long e double ficam no primeiro de dois índices, como na JVMS
- Custo: mais memória nos slots; arrays usam o tamanho real

Slot

```
typedef union {
    int32_t i;
    float   f;
    int64_t l;
    double  d;
    void   *ref;
} Slot;
```

A) SELEÇÃO DA INSTRUÇÃO

Do bytecode ao código

- O byte do opcode é o índice de uma tabela
- A posição guarda a função que executa a instrução
- Cada opcode é definido em um único lugar, para o exibidor e a execução
- Famílias como iload_0..3 compartilham o handler

tabela e handler

```
typedef void (*Handler)(VM *vm, Frame *f, u1 op);

typedef struct {
    const char *mnemonic;
    OperandKind kind;
    Handler     handler;
} OpcodeInfo;

[0x60] = {"iadd",    OPK_NONE, op_iadd},
[0x1a] = {"iload_0", OPK_NONE, op_iload_n},
[0x1b] = {"iload_1", OPK_NONE, op_iload_n},

void op_iload_n(VM *vm, Frame *f, u1 op) {
    push(f, f->locals[op - 0x1a]);
    f->pc += 1;
}
```

B) MEMÓRIA DINÂMICA

Heap com cabeçalho comum

- Objetos e arrays alocados com calloc
- A referência é o endereço, em slot.ref
- Limite: só a memória disponível
- Toda alocação entra em uma lista, liberada ao encerrar
- Sem coletor de lixo

ObjectHeader

```
typedef struct ObjectHeader {
    struct ObjectHeader *next_alloc;
    Class *class;
    u1    kind; /* objeto ou array */
} ObjectHeader;

/* ao alocar */
obj->next_alloc = vm->heap;
vm->heap = obj;
```

C) ÁREA DE MÉTODOS

Uma Class por classe

- Carga sob demanda, a superclasse antes
- Estados: carregada, ligada, inicializando, inicializada
- Code decodificado uma vez, na carga
- A primeira resolução do pool fica guardada
- invokevirtual e invokeinterface escolhem o método pela classe real do objeto

Class

```
typedef struct Class {
    const char   *name;
    ClassFile    *cf;
    struct Class *super;
    Method       *methods; /* Code decodificado */
    Field        *fields;  /* com o offset */
    Slot         *static_fields;
    u2           instance_slots;
    void        **resolved; /* por índice do pool */
    ClassState    state;
} Class;
```

D) ARRAYS

Um único bloco

- Elementos com o tamanho real do tipo
- A instrução faz o cast: iaload lê int32_t
- Matriz é array de arrays
- arraylength: desempilha, testa null e empilha a->length
- Todo acesso testa null e os limites

Array, com flexible array member

```
typedef struct {
    ObjectHeader hdr;
    u4      length;
    u1      atype;
    int64_t data[]; /* alinhamento; tipo vem da instrução */
} Array;

Array *a = calloc(1, sizeof(Array) + n * tam_elem);

[ cabeçalho | length | atype | e0 | e1 | e2 | ... ]
```

E) OBJETOS

Layout definido na carga

- Primeiro os campos da superclasse, depois os da classe
- O campo herdado fica na mesma posição em toda subclasse
- O offset resolvido vale para qualquer objeto
- Cada campo usa um Slot
- new zera os campos; o construtor roda via invokespecial

Object

```
typedef struct {
    ObjectHeader hdr;
    Slot fields[];
} Object;

// x na posição 0; em B, x em 0 (herdado) e y em 1
class A { int x; }
class B extends A { long y; }

/* getfield, depois da primeira resolução */
push(f, o→fields[field→offset]);
```

F) PILHA DE FRAMES

Um frame por chamada

1. invoke copia os argumentos para as locais do novo frame
2. O pc do chamador avança; o novo frame começa em pc = 0
3. return libera o frame e empilha o resultado no chamador
4. Sem frames, a execução termina

- push confere o limite (sp < limit)
- Contador de profundidade: StackOverflowError

Frame, em um único bloco

```
typedef struct Frame {
    struct Frame *caller;
    Class  *class;
    Method *method;
    u4      pc;
    Slot   *locals;
    Slot   *sp;
    Slot   *limit;
    Slot    slots[]; /* max_locals + max_stack */
} Frame;

slots: [ locais | pilha de operandos → ]
```

ESCOPO

Fora do escopo e dúvidas

Não será implementado

- Threads
- invokedynamic
- Coletor de lixo
- Verificação completa de bytecode

Parcial

- Exceções do programa: athrow e try/catch funcionam
- Exceções geradas pela JVM: imprime o erro e encerra
- Biblioteca simulada em C: System.out, String e StringBuffer

Dúvidas

1. O coletor de lixo é esperado?
2. Exceções da JVM precisam ser capturáveis?
3. Quais classes da biblioteca devem funcionar? Object pode ser simulada?
4. Threads podem ficar de fora?
5. É preciso rodar em 32 bits?
6. O exibidor deve continuar?

GRUPO

Atividades e divisão planejada

Nesta entrega: planejamos o projeto e fizemos os slides juntos.

Responsável	Parte	Inclui
Gabriel Pinto	Laço e instruções básicas (a)	Tabela de handlers; constantes, load/store, pilha, aritmética, conversões, comparações, desvios e switches
Antonio Coelho	Heap, arrays e objetos (b, d, e)	Alocação e liberação; new, newarray, anewarray, multianewarray, arraylength, getfield, putfield
Lucas Mendes	Área de métodos (c), coordenação	Carga, Code, layout dos campos, estados, clinit, resolução do pool, getstatic, putstatic
Leonardo Mileo	Frames e chamadas (f)	Criação de frames, invoke*, return, seleção de método virtual, athrow e tabela de exceções
Daniel Morais	Biblioteca, integração e testes	Modos run e view, System.out, String, StringBuffer, instanceof, checkcast, testes e analisadores